



LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II

Tugas 1 : RESTful API Murni dengan Express.js

Topik 8 : Posyandu : Data Balita

Disusun oleh:

Nama : Aisyah Ramadani

NIM : 2428240037

Kelas : SI5B

PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027

1. Tujuan Praktikum

Pratikum ini bertujuan agar mahasiswa mampu:

1. Membuat project berbasis `express.js` yang tersusun rapi (inisialisasi `npm`, `package.json` dengan script `start` dan `dev`, satu file `app.js` yang berkomentar), menjalankan secara lokal, mengunggahnya ke repository GitHub dengan commit bertahap, dan meen-deploy nya Vercel.
2. Membuat routing RESTful untuk resource `/toddlers` yang terdiri dari enam endpoint (route parameter `:id` dan filter dengan query string `req.query`) sesuai konvensi penamaan URL, serta middleware `catch-all` untuk menangani endpoint yang tidak terdaftar.
3. Menggunakan method HTTP GET, POST, PUT, dan DELETE sesuai fungsinya, disertai validasi field wajib, pembuatan id otomatis, dan penanganan kasus gagal (status 400 dan 404).
4. Menghasilkan response berformat JSON asli dengan struktur yang sesuai ketentuan (GET berupa data langsung: POST, PUT, DELETE, dan error berupa `{ status, message, data }`) beserta status code yang tepat.

2. Dasar Teori

A. REST dan Resource

REST (Representational State Transfer) merupakan gaya arsitektur yang digunakan dalam pembuatan layanan web dengan menjadikan data sebagai resource yang dapat diakses melalui URL (Fielding, 2000). Pada RESTful API, resource dikelola menggunakan method HTTP dan data dikembalikan dalam bentuk response, bukan berupa halaman web. Pada praktikum ini, resource yang digunakan adalah data balita Posyandu dengan endpoint `/toddlers`. Penamaan resource menggunakan kata benda jamak, huruf kecil, dan tidak menggunakan kata kerja.

B. Method HTTP

Method HTTP digunakan untuk menentukan operasi yang dilakukan terhadap suatu resource. GET digunakan untuk mengambil atau membaca data, POST digunakan untuk menambahkan data baru, PUT digunakan untuk memperbarui data yang sudah ada, sedangkan DELETE digunakan untuk menghapus data (Mozilla

Developer Network, n.d.-a). Method tersebut digunakan untuk menjalankan operasi CRUD, yaitu create, read, update, dan delete.

C. HTTP Status Code

HTTP status code merupakan kode yang menunjukkan hasil dari suatu request. Kode 200 menunjukkan bahwa request berhasil diproses, sedangkan 201 menunjukkan bahwa data baru berhasil dibuat. Kode 400 digunakan ketika request yang dikirim tidak valid, misalnya terdapat data wajib yang belum diisi. Sementara itu, kode 404 menunjukkan bahwa resource yang diminta tidak ditemukan (Mozilla Developer Network, n.d.-b).

D. JSON

JSON (JavaScript Object Notation) merupakan format yang digunakan untuk pertukaran data dan mudah dibaca oleh manusia maupun program. Data JSON disusun dalam bentuk pasangan key dan value. Pada API yang dibuat dalam praktikum ini, response dikirim dalam format JSON, termasuk ketika terjadi error. Response tersebut dikirim menggunakan `res.json()`.

E. Express.js

Express.js merupakan framework untuk Node.js yang digunakan untuk membuat aplikasi web dan API. Express menyediakan fitur seperti routing dan middleware (Express, n.d.). Routing digunakan untuk menentukan respons berdasarkan method dan URL yang diterima. Middleware `express.json()` digunakan agar data request dalam format JSON dapat dibaca melalui `req.body`. Selain itu, parameter yang terdapat pada URL dapat diakses melalui `req.params`, sedangkan query string dapat diakses melalui `req.query`. Node.js sendiri merupakan runtime yang digunakan untuk menjalankan JavaScript pada sisi server (OpenJS Foundation, n.d.).

3. Alat dan Bahan

No	Nama Alat / Bahan	Versi / Keterangan
1	Node.js (LTS)	Versi : v22.14.0
2	Npm	Versi : 12.0.2
3	Express.js	Versi 5.2.1 (Framework web)

4	Nodemon	Versi 3.1.14 (devDependency, restart otomatis)
5	Visual Studio Code	Editor code
6	Postman	Alat pengujian endpoint API
7	Git	Versi 2.56.0. Windows.1 (version control)
8	GitHub	Akun GitHub untuk repository publik
9	Vercel	Akun vercel untuk deploy

4. Langkah Praktikum

a. Inisialisasi Project

Pada tahap awal, project dibuat menggunakan perintah `npm init -y` yang menghasilkan file `package.json`. File tersebut digunakan untuk menyimpan informasi dan konfigurasi project Node.js, seperti nama project, versi, script, serta package yang digunakan. Selanjutnya dilakukan instalasi Express.js sebagai framework untuk membuat REST API dan Nodemon sebagai tools yang membantu menjalankan ulang server secara otomatis ketika terdapat perubahan pada kode.

```

{} package.json > {} dependencies > abc express
1  {
2    "name": "tugas1_restful_2428240037_aisyahramadani",
3    "version": "1.0.0",
4    "description": "Tugas 1 restful API Express.js - Topik 8 Data Balita Posyandu",
5    "main": "app.js",
6    >Debug
7    "scripts": {
8      "start": "node app.js",
9      "dev": "nodemon app.js"
10   },
11   "keywords": [],
12   "type": "commonjs",
13   "dependencies": {
14     "express": "^5.2.1"
15   },
16   "devDependencies": {
17     "nodemon": "^3.1.14"
18   }

```

Gambar 1. Isi file `package.json`

Selanjutnya membuat `File.gitignore` dibuat agar `node_modules/` (dapat dibuat ulang dengan `npm install`) dan `.env` (berisi data rahasia) tidak ikut diunggah ke GitHub. Setelah itu repository Git diinisialisasi dan dilakukan commit pertama.

`node_modules/`

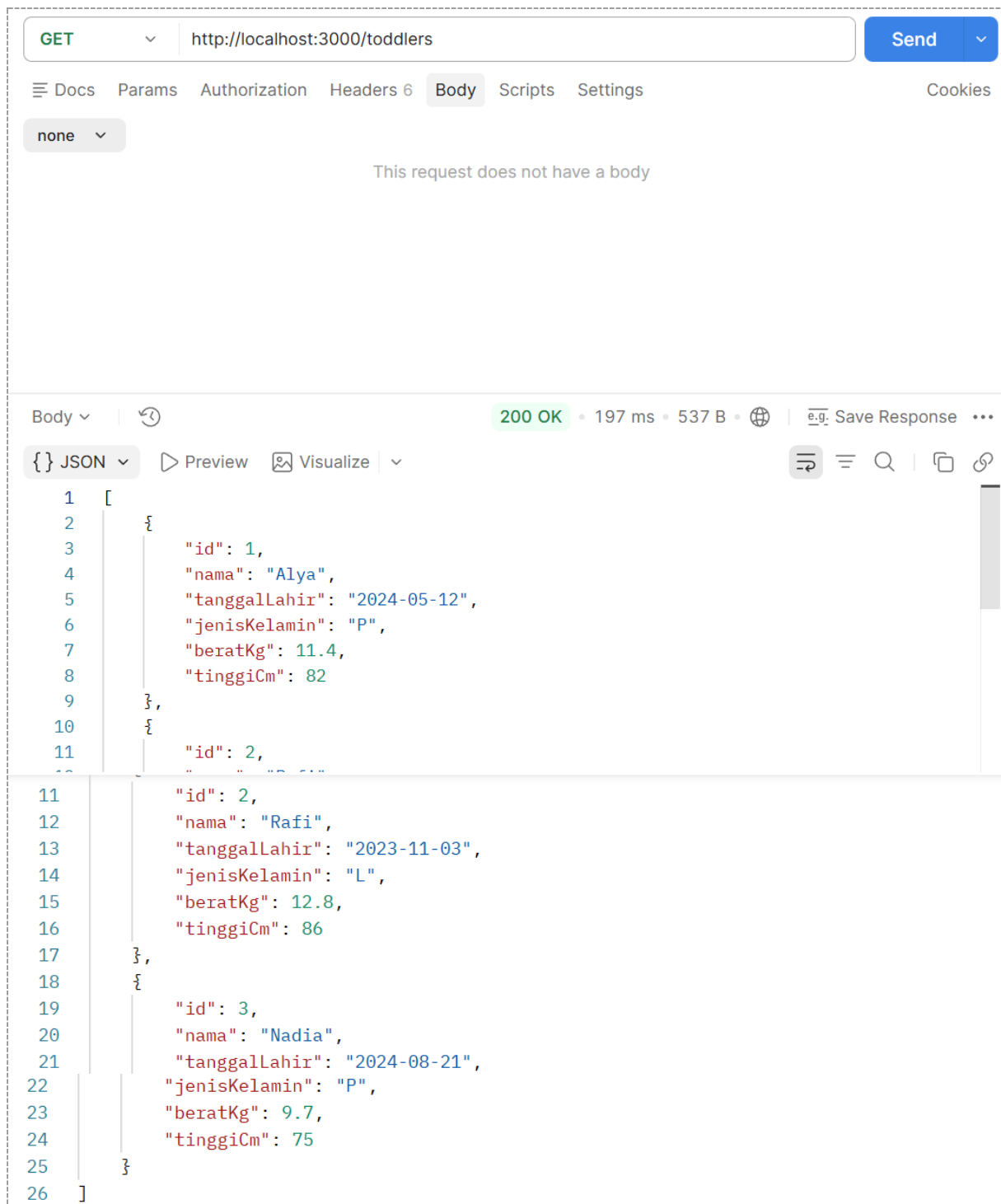
```
.env
git init
git add .
git commit -m "init project express"
```

b. Data Awal dan Route GET/

File `app.js` dibuat dengan urutan: impor Express, middleware `express.json()`, array data awal (3 data balita) beserta variabel `nextId` untuk id berikutnya, lalu route GET `/` yang membalas JSON berisi nama, NIM, nomor topik, dan daftar endpoint.

```
js app.js > ...
1  const express = require('express');
2  const app = express();
3  app.use(express.json());
4
5  let toddlers = [
6    { id: 1, nama: 'Alya', tanggalLahir: '2024-05-12', jenisKelamin: 'P', beratKg: 11.4, tinggiCm: 82 },
7    { id: 2, nama: 'Rafi', tanggalLahir: '2023-11-03', jenisKelamin: 'L', beratKg: 12.8, tinggiCm: 86 },
8    { id: 3, nama: 'Nadia', tanggalLahir: '2024-08-21', jenisKelamin: 'P', beratKg: 9.7, tinggiCm: 75 },
9  ];
10
11 let nextId = 4;
12
13 app.get('/', (req, res) => {
14   res.json({
15     nama: 'Aisyah Ramadani',
16     nim: '2428240037',
17     topik: 8,
18     namaTopik: 'Posyandu - Data Balita',
19     endpoints: [
20       'GET /toddlers',
21       'GET /toddlers/:id',
22       'POST /toddlers',
23       'PUT /toddlers/:id',
24       'DELETE /toddlers/:id',
25       'GET /toddlers?jenisKelamin=L|P',
26     ],
27   });
28 });
```

Gambar 2. Kode data awal dan route GET / pada `app.js`



Gambar 3. Server berjalan dan hasil GET / (info API)

c. Route GET (Semua Data, Satu Data, dan Filter)

GET /toddlers mengembalikan seluruh data dan, bila ada query string `?jenisKelamin=`, menyaring data dengan filter (hasil kosong berupa array `[]` dengan status 200). GET /toddlers/:id membaca id dari `req.params`, mengubahnya ke angka dengan `parseInt`, lalu mencarinya dengan `find`; jika

tidak ditemukan dikirim status 404. Response GET berupa data langsung tanpa status dan message.

```
30 // GET /toddlers
31 // Ambil semua data. Bisa difilter: GET /toddlers?jenisKelamin=P
32 app.get('/toddlers', (req, res) => {
33   const { jenisKelamin } = req.query;
34
35   // kalau ada filter, saring datanya
36   if (jenisKelamin) {
37     const hasil = toddlers.filter((t) => t.jenisKelamin === jenisKelamin);
38     return res.json(hasil);
39   }
40
41   // kalau tidak ada filter, kirim semua data
42   res.json(toddlers);
43 });
44
45 // GET /toddlers/:id
46 // Ambil satu data berdasarkan id
47 app.get('/toddlers/:id', (req, res) => {
48   const id = parseInt(req.params.id);
49   const toddler = toddlers.find((t) => t.id === id);
50
51   // data tidak ada -> 404
52   if (!toddler) {
53     return res.status(404).json({
54       status: 'error',
55       message: `Data dengan id ${id} tidak ditemukan`,
56       data: null,
57     });
58   }
59 }
```

Gambar 4. Kode route GET /toddlers dan GET /toddlers/:id

d. Route POST dan Validasi

POST /toddlers membaca data dari req.body. Server memvalidasi field wajib (nama, tanggalLahir, jenisKelamin, beratKg), nilai jenisKelamin yang hanya boleh L atau P, dan tipe beratKg yang harus angka;

jika gagal dikirim status 400. Jika lolos, id dibuat otomatis dengan `nextId++`, data dimasukkan ke array, dan dikirim status 201.

```
63 // POST /toddlers
64 // Body: { "nama": "Citra", "tanggalLahir": "2024-01-15", "jenisKelamin": "P", "beratKg": 10.2, "tinggiCm": 79 }
65 app.post('/toddlers', (req, res) => {
66   const { nama, tanggalLahir, jenisKelamin, beratKg, tinggiCm } = req.body;
67
68   // validasi: field wajib tidak boleh kosong -> 400
69   if (!nama || !tanggalLahir || !jenisKelamin || beratKg === undefined) {
70     return res.status(400).json({
71       status: 'error',
72       message: 'nama, tanggalLahir, jenisKelamin, dan beratKg wajib diisi',
73       data: null,
74     });
75   }
76
77   // validasi: jenisKelamin hanya boleh L atau P -> 400
78   if (jenisKelamin !== 'L' && jenisKelamin !== 'P') {
79     return res.status(400).json({
80       status: 'error',
81       message: 'jenisKelamin harus L atau P',
82       data: null,
83     });
84   }
85
86   // validasi: beratKg harus angka -> 400
87   if (typeof beratKg !== 'number') {
88     return res.status(400).json({
89       status: 'error',
90       message: 'beratKg harus berupa angka',
91       data: null,
92     });
93   }
94
95   // buat data baru, id otomatis dari nextId
96   const baru = { id: nextId++, nama, tanggalLahir, jenisKelamin, beratKg, tinggiCm };
97   toddlers.push(baru);
98
99   // berhasil -> 201 + data yang baru dibuat
100  res.status(201).json({
101    status: 'success',
102    message: 'Data berhasil ditambahkan',
103    data: baru,
104  });
105 });
```

Gambar 5. Kode route POST /toddlers beserta validasinya

e. Route PUT dan DELETE

PUT /toddlers/:id mengganti seluruh data (penggantian penuh) dengan id yang tetap. Urutan pemeriksaannya: id dicari lebih dulu (404 jika tidak ada), kemudian body divalidasi seperti pada POST (400 jika tidak valid). DELETE /toddlers/:id mencari posisi data dengan `findIndex`, menghapusnya dengan `splice`, dan membalas status 200 dengan `data: null` dan pesan yang menyebut id yang dihapus; jika id tidak ada dikirim status 404.

```

107 // PUT /toddlers/:id
108 // Body: { "nama": "Alya Putri", "tanggalLahir": "2024-05-12", "jenisKelamin": "P", "beratKg": 12, "tinggiCm": 84 }
109 app.put('/toddlers/:id', (req, res) => {
110   const id = parseInt(req.params.id);
111   const index = toddlers.findIndex((t) => t.id === id);
112
113   // data tidak ada -> 404
114   if (index === -1) {
115     return res.status(404).json({
116       status: 'error',
117       message: `Data dengan id ${id} tidak ditemukan`,
118       data: null,
119     });
120   }
121
122   const { nama, tanggalLahir, jenisKelamin, beratKg, tinggiCm } = req.body;
123
124   // ganti seluruh data lama dengan data baru (id tetap)
125   toddlers[index] = { id, nama, tanggalLahir, jenisKelamin, beratKg, tinggiCm };
126
127   res.json({
128     status: 'success',
129     message: 'Data berhasil diubah',
130     data: toddlers[index],
131   });
132 });
133
134 // DELETE /toddlers/:id
135 app.delete('/toddlers/:id', (req, res) => {
136   const id = parseInt(req.params.id);
137   const index = toddlers.findIndex((t) => t.id === id);
138
139   // data tidak ada -> 404
140   if (index === -1) {
141     return res.status(404).json({
142       status: 'error',
143       message: `Data dengan id ${id} tidak ditemukan`,
144       data: null,
145     });
146   }
147
148   // hapus 1 data pada posisi index
149   toddlers.splice(index, 1);
150
151   res.json({
152     status: 'success',
153     message: `Data balita dengan id ${id} berhasil dihapus`,
154     data: null,
155   });
156 });

```

Gambar 6. Kode route PUT dan DELETE pada app.js

f. Middleware catch-all 404 dan Penutup app.js

Middleware `app.use` tanpa alamat ditempatkan setelah semua route sehingga menangkap setiap request ke alamat yang tidak terdaftar dan membalas JSON dengan status 404. Di bagian paling bawah, `app.listen` hanya dijalankan saat

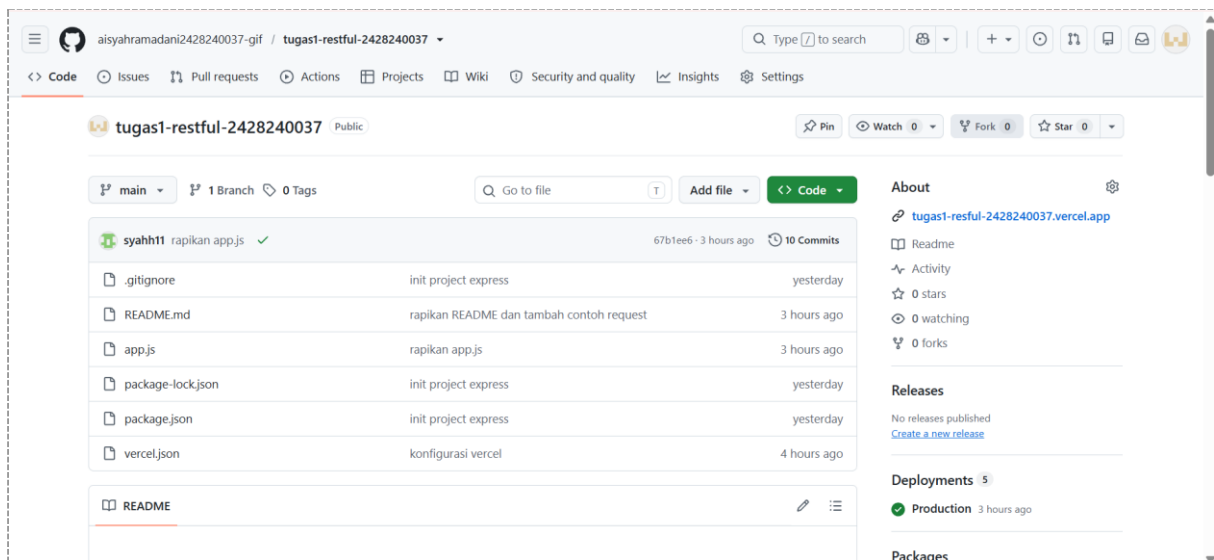
lokal dan aplikasi diekspor dengan `module.exports` agar dapat dijalankan Vercel (serverless).

```
185 // Catch-all 404: harus paling bawah, setelah semua route
186 app.use((req, res) => {
187   res.status(404).json({
188     status: 'error',
189     message: 'Endpoint tidak ditemukan',
190     data: null,
191   });
192 });
193
194
195 const PORT = process.env.PORT || 3000;
196 if (process.env.NODE_ENV !== 'production') {
197   app.listen(PORT, () => console.log(`Server berjalan di http://localhost:${PORT}`));
198 }
199 module.exports = app;
```

Gambar 7. Kode middleware catch-all 404 dan bagian akhir `app.js`

g. Push ke GitHub

Repository publik bernama `tugas1-restful-2428240037` dibuat di GitHub tanpa README bawaan, kemudian project diunggah. Commit dilakukan bertahap pada setiap tahap pengerjaan (init project, server dan route GET /, POST dan validasi, PUT dan DELETE, README, konfigurasi Vercel).

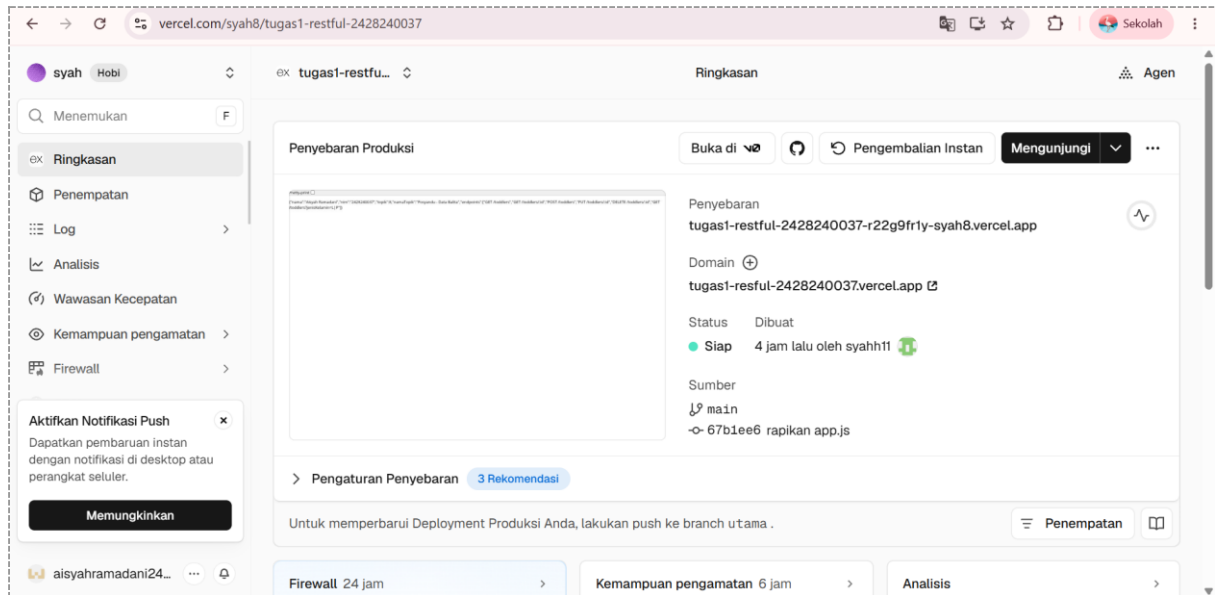


Gambar 8. Halaman repository GitHub `tugas1-restful-2428240037`

h. Deploy ke Vercel

File `vercel.json` ditambahkan di root project agar `app.js` dijalankan sebagai fungsi serverless dan semua alamat diarahkan ke file tersebut. Repository kemudian diimpor melalui dashboard Vercel (Add New, Project) lalu di-deploy. Setiap push

ke branch main akan otomatis ter-deploy ulang. <https://tugas1-resful-2428240037.vercel.app/>



Gambar 9. Deploy berhasil di dashboard Vercel

i. Hasil Pengujian

Pengujian dilakukan dengan Post Postman pada URL Vercel: [<https://tugas1-resful-2428240037.vercel.app/>]. Tabel berikut memuat 10 skenario pengujian beserta satu skenario tambahan untuk middleware catch-all.

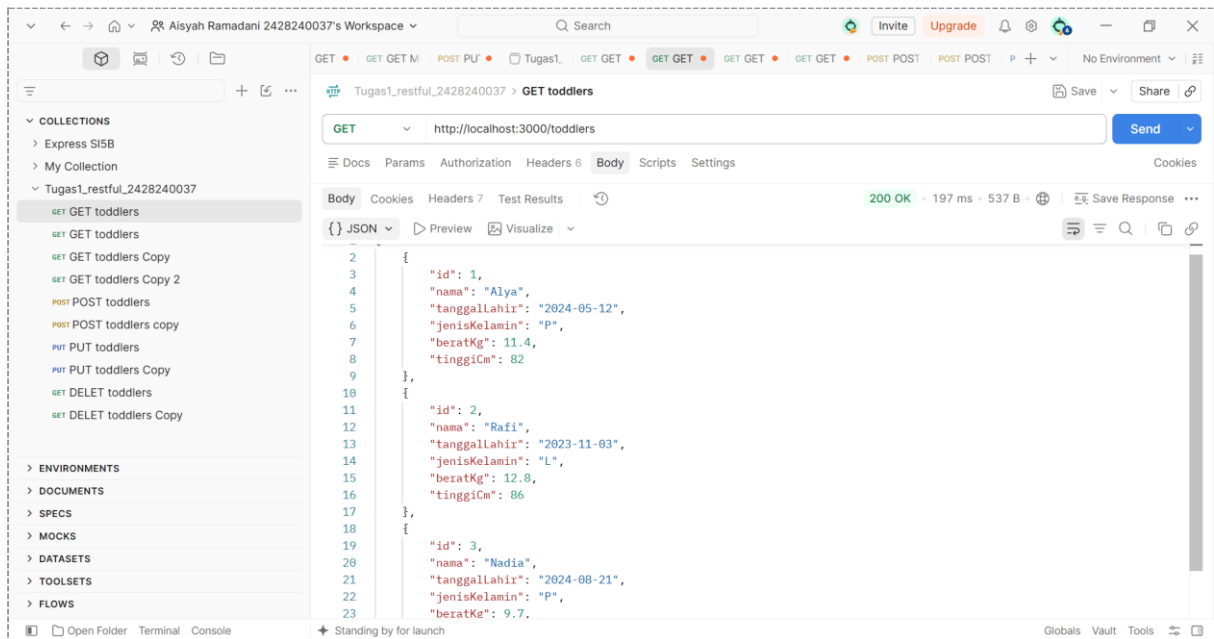
No	Method	Endpoint	Data request	Status diharapkan	Status hasil	Keterangan
1	GET	/toddlers	—	200	200	Menampilkan seluruh data balita
2	GET	/toddlers/1	—	200	200	Menampilkan data balita ID 1
3	GET	/toddlers/99	—	404	404	ID tidak ditemukan
4	GET	/toddlers?jenisKelamin=P	—	200	200	Menampilkan data balita dengan jenis kelamin p
5	POST	/toddlers	Body lengkap	201	201	Menambahkan data balita

No	Method	Endpoint	Data request	Status diharapkan	Status hasil	Keterangan
6	POST	/toddlers	Field wajib kosong	400	400	Menolak data yang tidak lengkap
7	PUT	/toddlers/1	Body lengkap	200	200	Memperbarui data ID 1
8	PUT	/toddlers/99	Body lengkap	404	404	ID tidak ditemukan
9	DELETE	/toddlers/1	—	200	200	Menghapus data ID 1
10	DELETE	/toddlers/99	—	404	404	ID tidak ditemukan
11	GET	/abc (catch-all)	—	404	404	Endpoint tidak di temukan

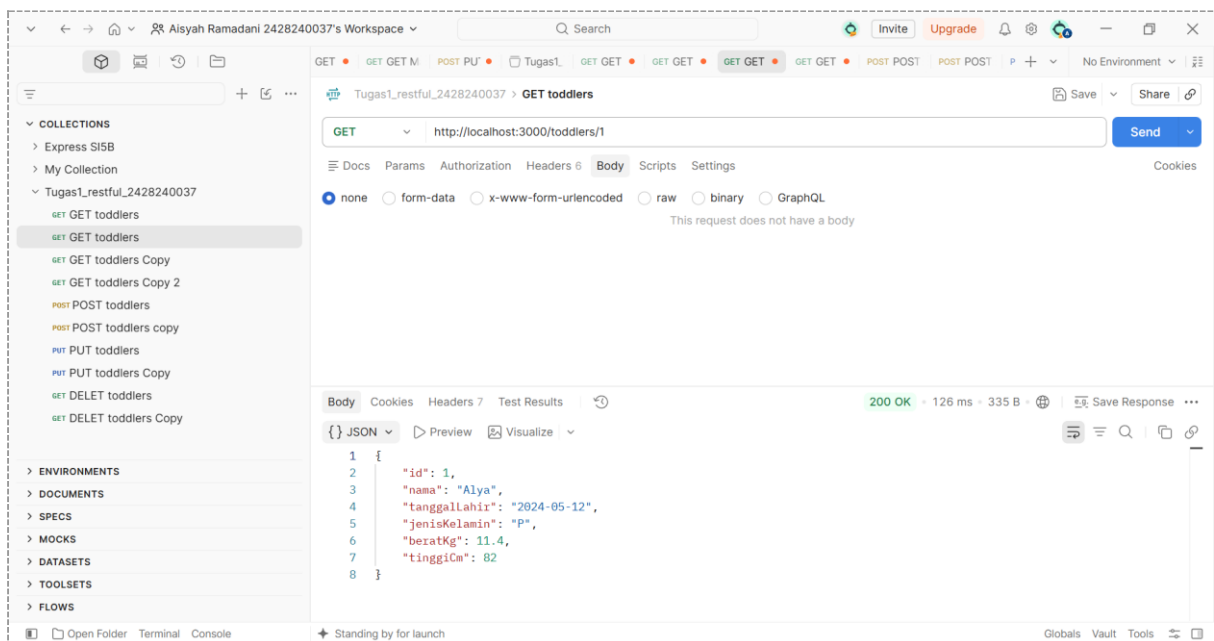
Data request yang dipakai pada pengujian:

- POST berhasil (no. 5): {"nama":"Citra","tanggalLahir":"2024-01-15","jenisKelamin":"P","beratKg":10.2,"tinggiCm":79}
- POST field wajib kosong (no. 6): {"tanggalLahir":"2024-01-15","jenisKelamin":"P","beratKg":10.2} (field nama tidak dikirim)
- PUT (no. 7 dan 8): {"nama":"Alya Putri","tanggalLahir":"2024-05-12","jenisKelamin":"P","beratKg":12,"tinggiCm":84}

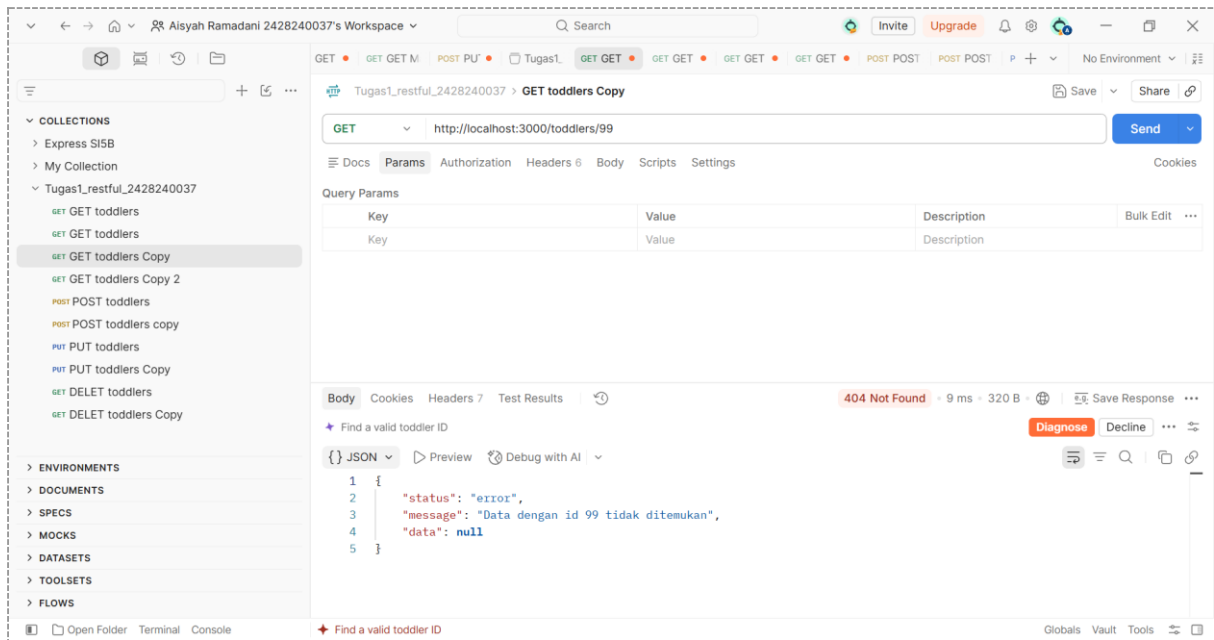
Screenshot setiap skenario (memuat method, URL, body request untuk POST/PUT, status code, dan body response):



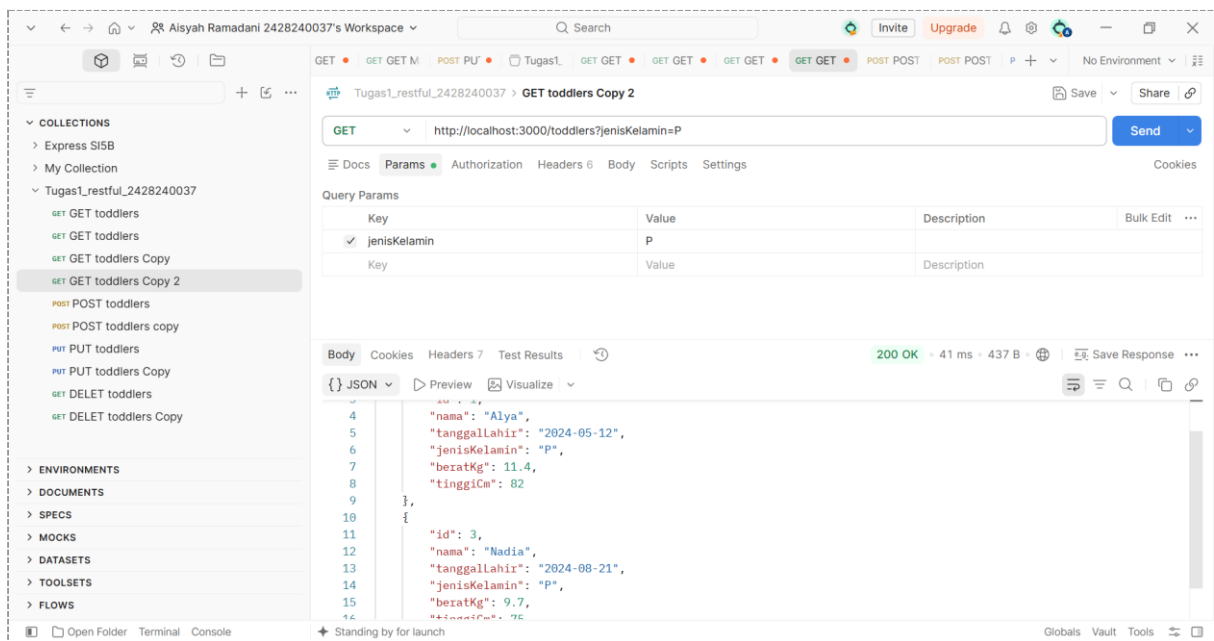
Gambar 10. Skenario 1: GET /toddlers (200)



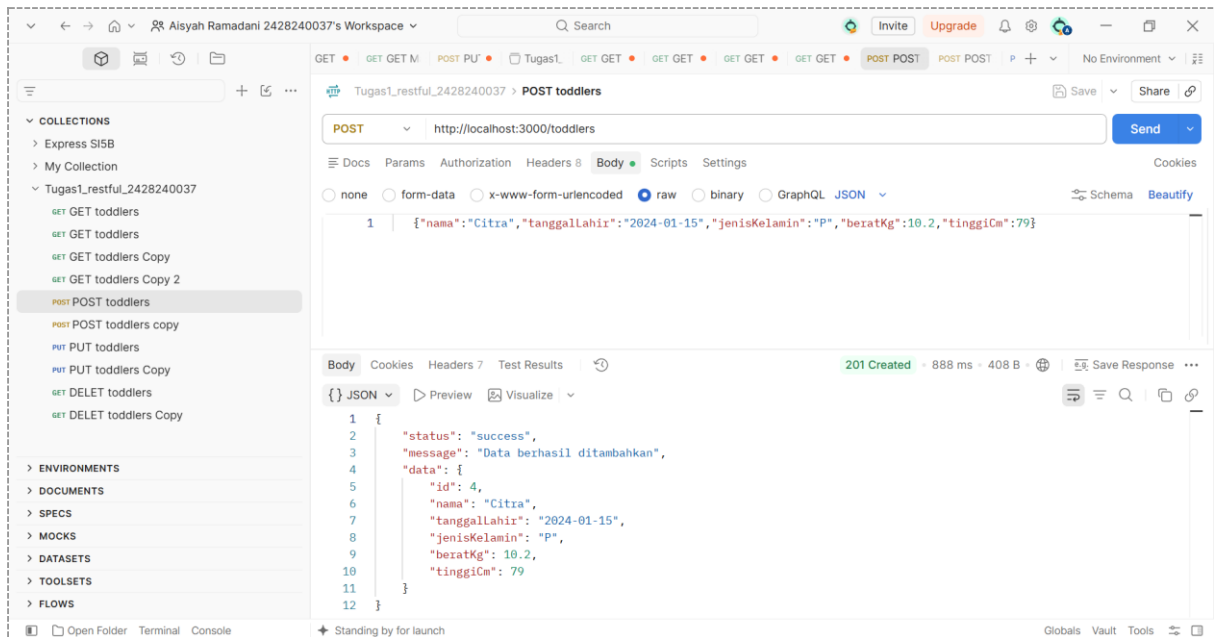
Gambar 11. Skenario 2: GET /toddlers/1 (200)



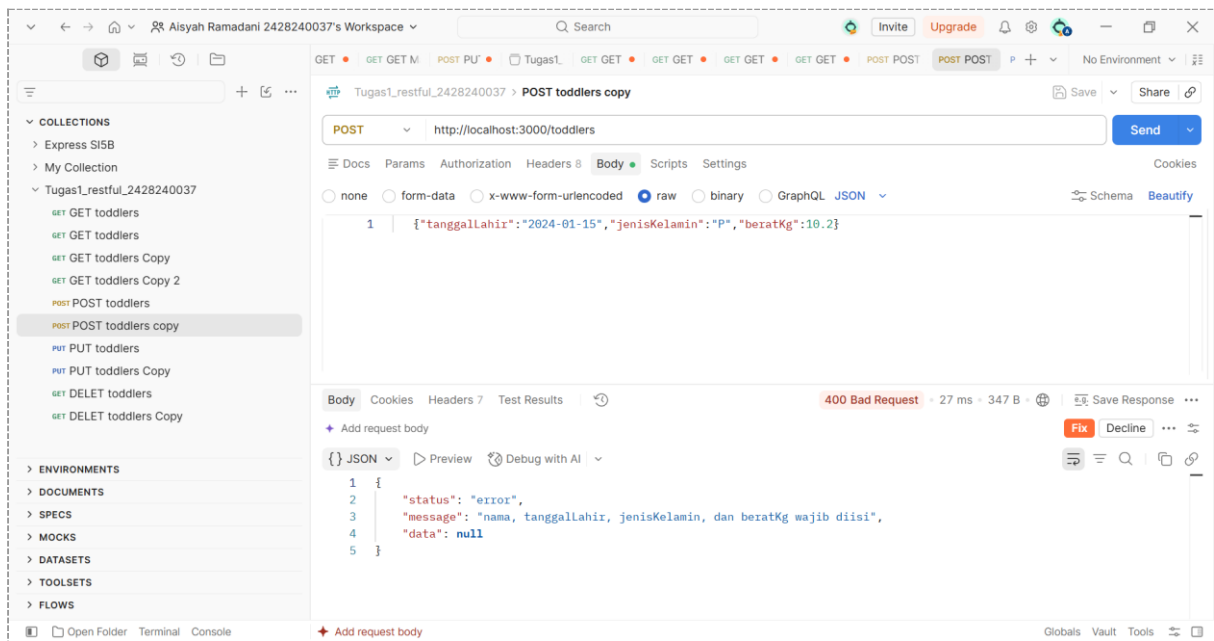
Gambar 12. Skenario 3: GET /toddlers/99 (404)



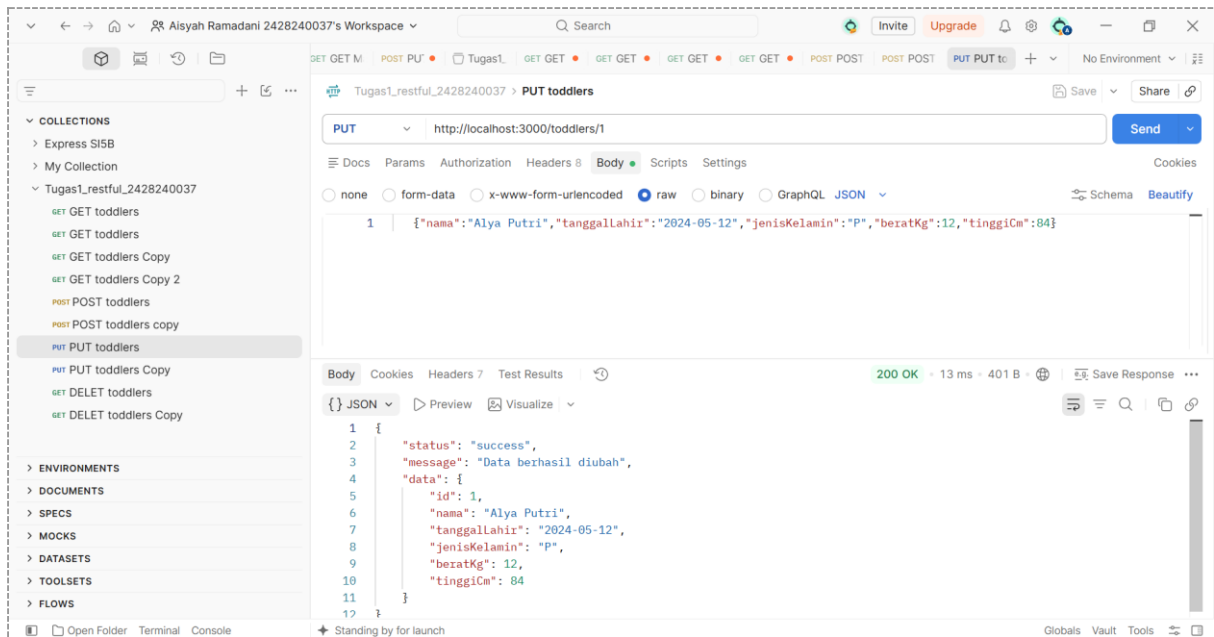
Gambar 13. Skenario 4: GET /toddlers?jenisKelamin=P (200)



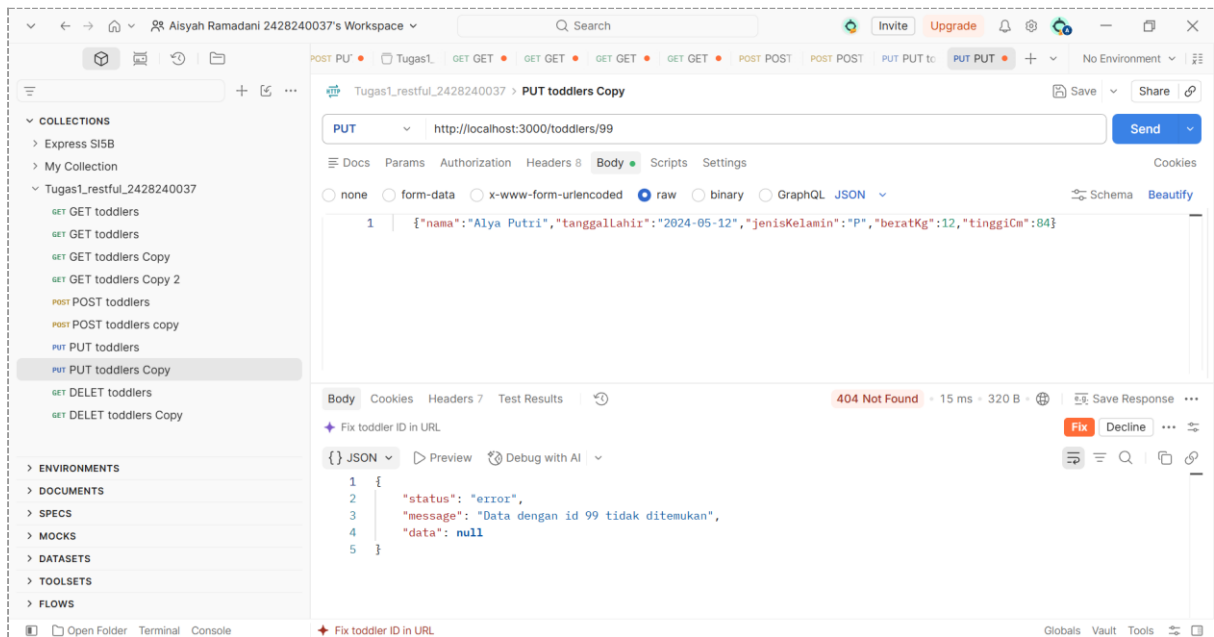
Gambar 14. Skenario 5: POST /toddlers dengan body lengkap (201)



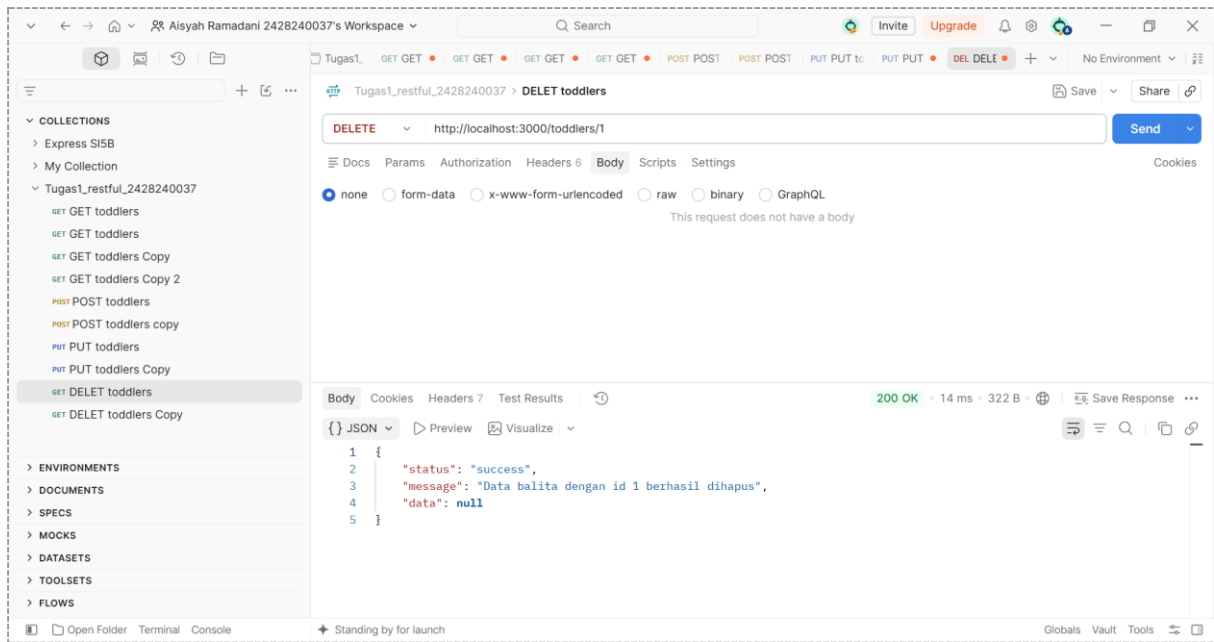
Gambar 15. Skenario 6: POST /toddlers dengan field wajib kosong (400)



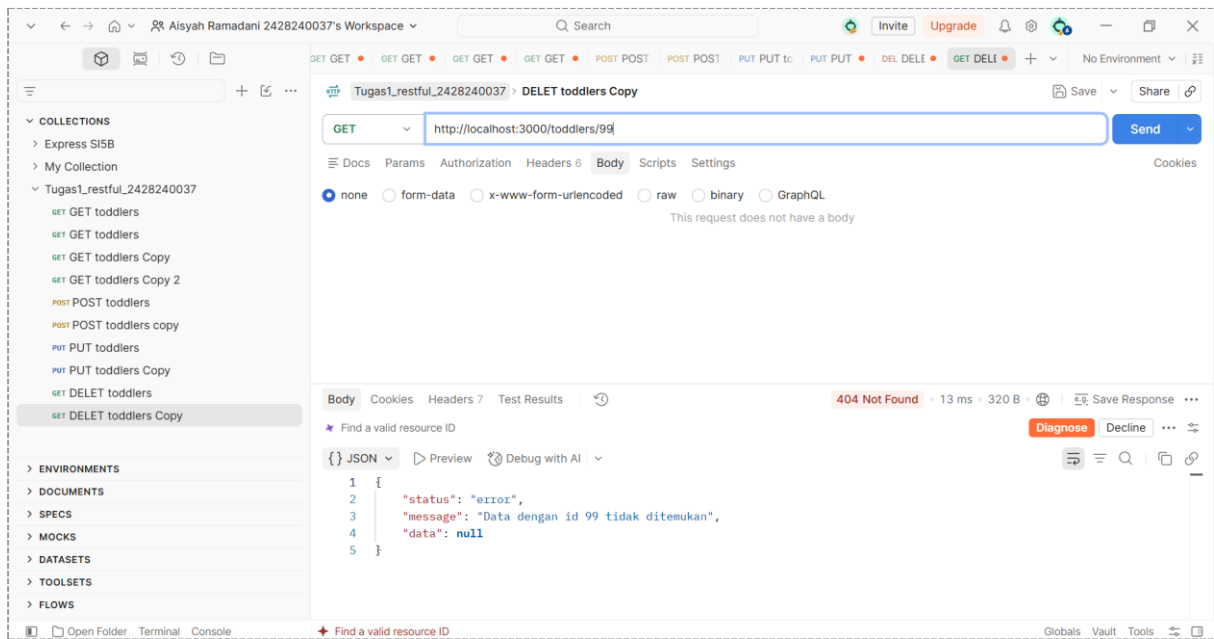
Gambar 16. Skenario 7: PUT /toddlers/1 dengan body lengkap (200)



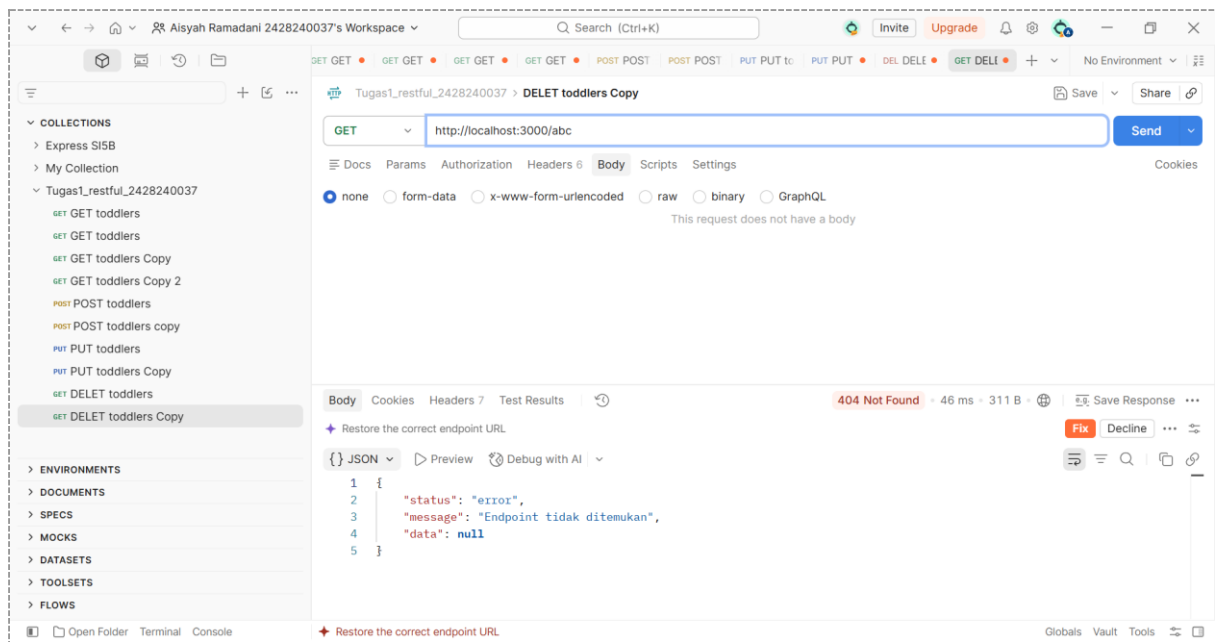
Gambar 17. Skenario 8: PUT /toddlers/99 dengan body lengkap (404)



Gambar 18. Skenario 9: DELETE /toddlers/1 (200)



Gambar 19. Skenario 10: DELETE /toddlers/99 (404)



Gambar 20. Skenario 11: GET /abc, endpoint tidak terdaftar (404)

j. Pembahasan

a. Alasan pemilihan Method dan status Code

- **GET (200)** dipilih untuk membaca data karena sifatnya hanya mengambil dan tidak mengubah array. Jika id tidak ditemukan dikirim **404**, sedangkan filter yang tidak cocok tetap **200** dengan array kosong karena permintaannya valid.
- **POST (201)** dipilih untuk menambah data baru. Status 201 menandakan resource baru berhasil dibuat, berbeda dari 200 yang hanya berarti permintaan berhasil. Jika field wajib kosong atau nilainya tidak sesuai dikirim **400** karena kesalahan ada pada data yang dikirim klien.
- **PUT (200)** dipilih untuk mengganti seluruh data pada id tertentu. Karena penggantian penuh, semua field wajib harus dikirim kembali; jika tidak lengkap dikirim **400**, dan jika id tidak ada dikirim **404**.
- **DELETE (200)** dipilih untuk menghapus data. Response berisi pesan dan `data: null` sehingga klien mengetahui data mana yang terhapus; jika id tidak ada dikirim **404**.
- **Catch-all (404)** menangani alamat yang tidak terdaftar dengan response JSON sehingga API tidak pernah membalas halaman HTML bawaan Express.

b. Kendala dan Solusi

No	Kendala	Solusi
1	File .gitignore tertulis tanpa titik di depan namanya (gitignore) sehingga Git tidak mengabaikan node_modules.	Mengganti nama file menjadi .gitignore sebelum commit pertama dilakukan.
2	Request filter yang ditulis /toddlers/jenisKelamin=P menghasilkan pesan "Data dengan id NaN tidak ditemukan" karena dibaca sebagai route /toddlers/:id.	Menulis filter dengan tanda tanya: /toddlers?jenisKelamin=P sehingga dibaca sebagai query string.
3	Data kembali ke 3 data awal setiap server restart (nodemon) dan dapat kembali ke data awal di Vercel karena data hanya disimpan di array memori.	Hal ini wajar pada penyimpanan di memori dan sifat serverless. Pengujian dilakukan berurutan, dan setiap request tetap mendapat response yang benar.

k. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Project Express.js berhasil dibuat dalam satu file `app.js` dengan script `start` dan `dev`, berjalan secara lokal, diunggah ke GitHub dengan commit bertahap, dan di-deploy ke Vercel (tujuan 1).
2. Enam endpoint RESTful untuk resource `/toddlers` berhasil dibuat, termasuk route parameter `:id`, filter query string `?jenisKelamin=`, dan middleware `catch-all 404` (tujuan 2).
3. Method GET, POST, PUT, dan DELETE berfungsi sesuai tugasnya, dengan validasi field wajib, id otomatis, serta penanganan kasus gagal berstatus 400 dan 404 (tujuan 3).
4. Seluruh response berformat JSON murni dengan struktur dan status code yang sesuai ketentuan, yaitu 200, 201, 400, dan 404 (tujuan 4).

Daftar Pustaka

Express. (n.d.). *Express: Fast, unopinionated, minimalist web framework for Node.js*.
<https://expressjs.com/>

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine].

Mozilla Developer Network. (n.d.-a). *HTTP request methods*. MDN Web Docs.
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>

Mozilla Developer Network. (n.d.-b). *HTTP response status codes*. MDN Web Docs.
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>

OpenJS Foundation. (n.d.). *Node.js documentation*. <https://nodejs.org/docs/latest/api/>

Lampiran

Link repository GitHub :

[ISI LINK GITHUB : <https://github.com/aisyahramadani2428240037-gif/tugas1-restful-2428240037>]

Link deploy Vercel :

[ISI LINK VERCEL : <https://tugas1-resful-2428240037.vercel.app/>]

Riwayat commit (git log --online) :

```
PS D:\Tugas1_restful_2428240037_AisyahRamadani> git log --online main
67b1ee6 (HEAD -> main, origin/main) rapikan app.js
3a39f7f rapikan README dan tambah contoh request
:
67b1ee6 (HEAD -> main, origin/main) rapikan app.js
3a39f7f rapikan README dan tambah contoh request
079476f konfigurasi vercel
030be28 Add Vercel deployment link
621c1a6 Update README
357e980 Initial project setup
5c11ef9 tambah PUT dan DELETE
5648216 tambah POST dan validasi
476c1a5 tambah server dan route GET /
2210d23 init project express
```

Gambar 21. Hasil perintah git log --online